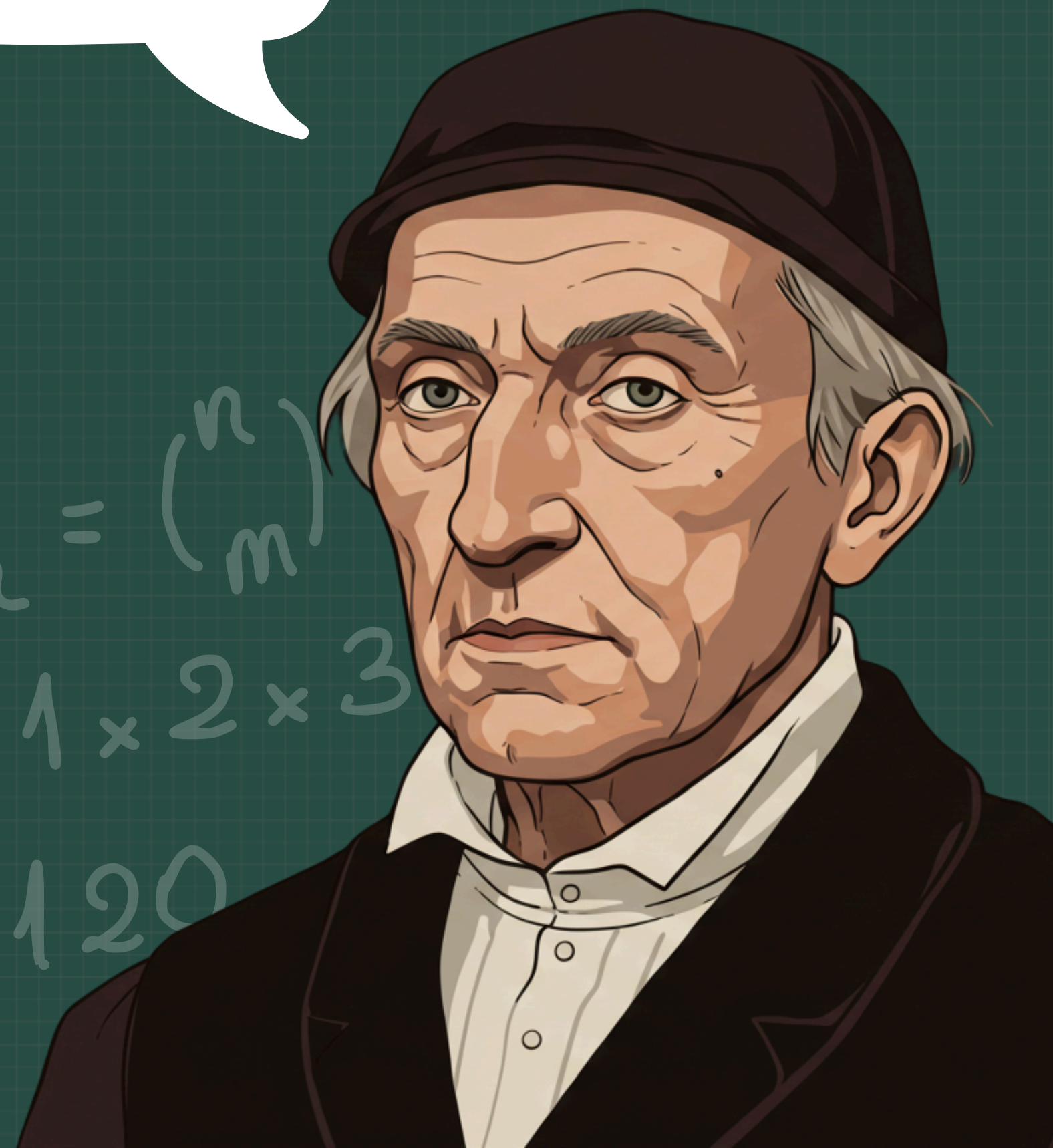


TEORÍA DE NÚMEROS

$$1+2+\dots+n = \frac{n(n+1)}{2}$$

- Divisibilidad
- Funciones
- Primalidad
- Factorización
- M.C.D. y M.C.M.

$$C_m^n = \binom{n}{m}$$
$$3! = 1 \times 2 \times 3$$
$$5! = 120$$



NÚMERO FALTANTE

Te dan todos los números enumerados desde el 1 hasta el n excepto uno.
Tu tarea es encontrar el número que falta.

Imaginemos que $n = 5$

1

2

3

4

5



<https://cses.fi/problemset/task/1083>

NÚMERO FALTANTE

Te dan todos los números enumerados desde el 1 hasta el n excepto uno.
Tu tarea es encontrar el número que falta.

Imaginemos que $n = 5$

1

2

3

5



<https://cses.fi/problemset/task/1083>

SUMA DE NÚMEROS CONSECUTIVOS DE GAUSS

Imaginemos que $n = 5$

$$\frac{5 \cdot 6}{2} = 15$$

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$



NÚMERO FALTANTE

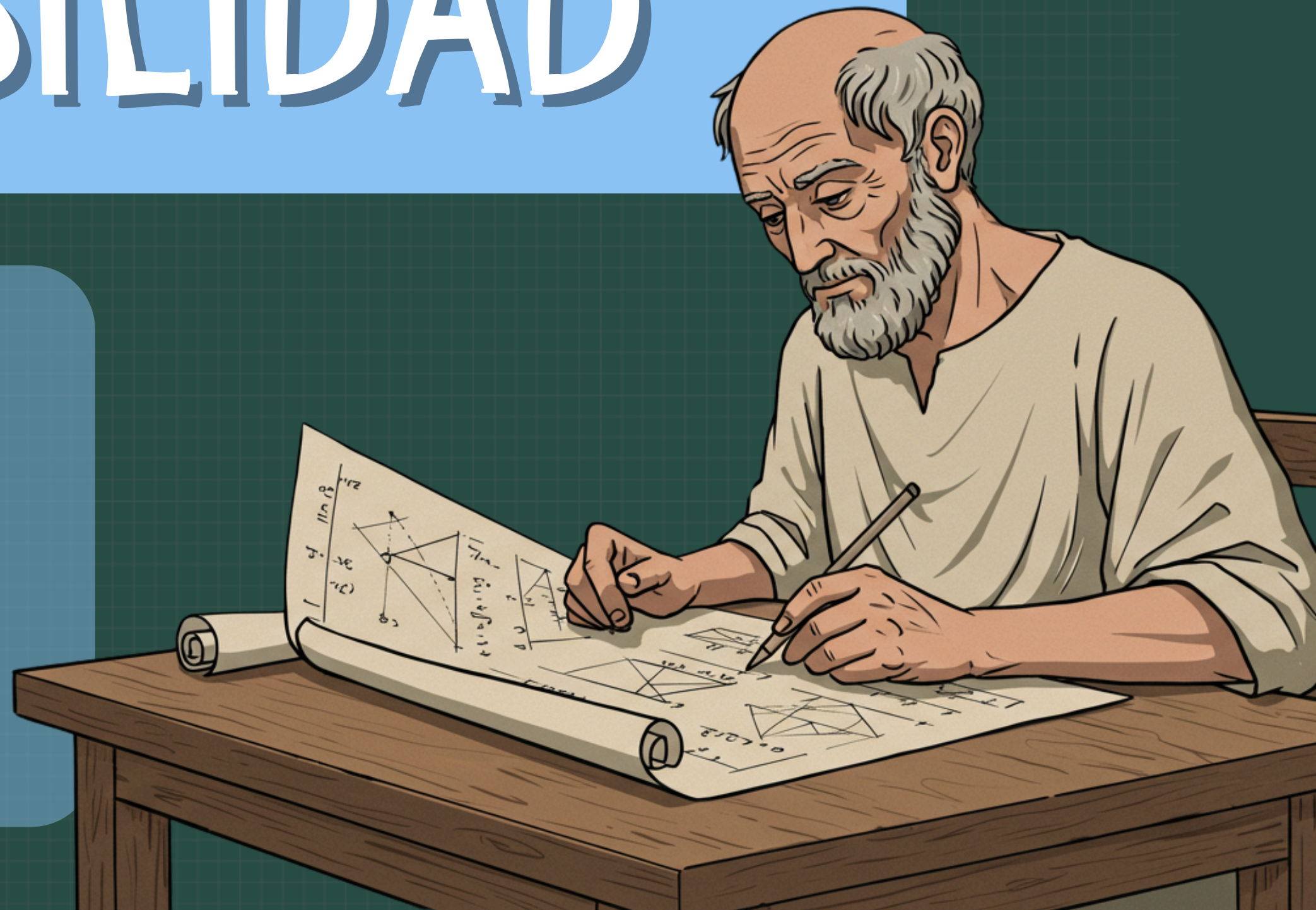
Te dan todos los números enumerados desde el 1 hasta el n excepto uno.
Tu tarea es encontrar el número que falta.

Imaginemos que $n = 5$

$$1 + 2 + 3 + 5 = 15 - 11 = 4$$

DIVISIBILIDAD

La divisibilidad es una propiedad matemática que indica si un número se puede dividir entre otro sin dejar residuo



DIVISIBILIDAD

- Sean a y b números enteros (a es diferente de 0).
- Decimos que « a es divisor de b » si existe un entero k tal que: $a \times k = b$
- Se representa como: $a \mid b$
- Ejemplos:
 - $-5 \mid 15$, porque $-5 \times -3 = 15$
 - $1 \mid -13$, porque $1 \times -13 = -13$
 - $8 \mid 0$, porque $8 \times 0 = 0$
 - $4 \mid 36$, porque $4 \times 9 = 36$

DIVISORES DE UN ENTERO N

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int N = 1000;
7
8      vector<int> divisores;
9      for (int div = 1; div ≤ N; div++) {
10         if (N % div == 0) {
11             divisores.push_back(div);
12         }
13     }
14
15     cout << divisores.size() << endl;
16     for (int i = 0; i < divisores.size(); i++) {
17         cout << divisores[i] << " ";
18     }
19
20     return 0;
21 }
```

1º Forma: Iterar todos los números del 1 al N y verificar cuál de ellos es divisor de este número.

Desventaja: Si el número es muy grande, el programa nos puede dar **Time Limit Exceeded** por tener una complejidad algorítmica: **O(N)**

DIVISORES DE UN ENTERO N

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      long long N = 10000000000000;
7
8      vector<long long> divisores;
9      for (long long div = 1; div * div ≤ N; div++) {
10         if (N % div == 0) {
11             divisores.push_back(div);
12             if (N / div ≠ div) {
13                 divisores.push_back(N / div);
14             }
15         }
16     }
17
18     cout << divisores.size() << endl;
19     for (int i = 0; i < divisores.size(); i++) {
20         cout << divisores[i] << " ";
21     }
22
23     return 0;
24 }
```

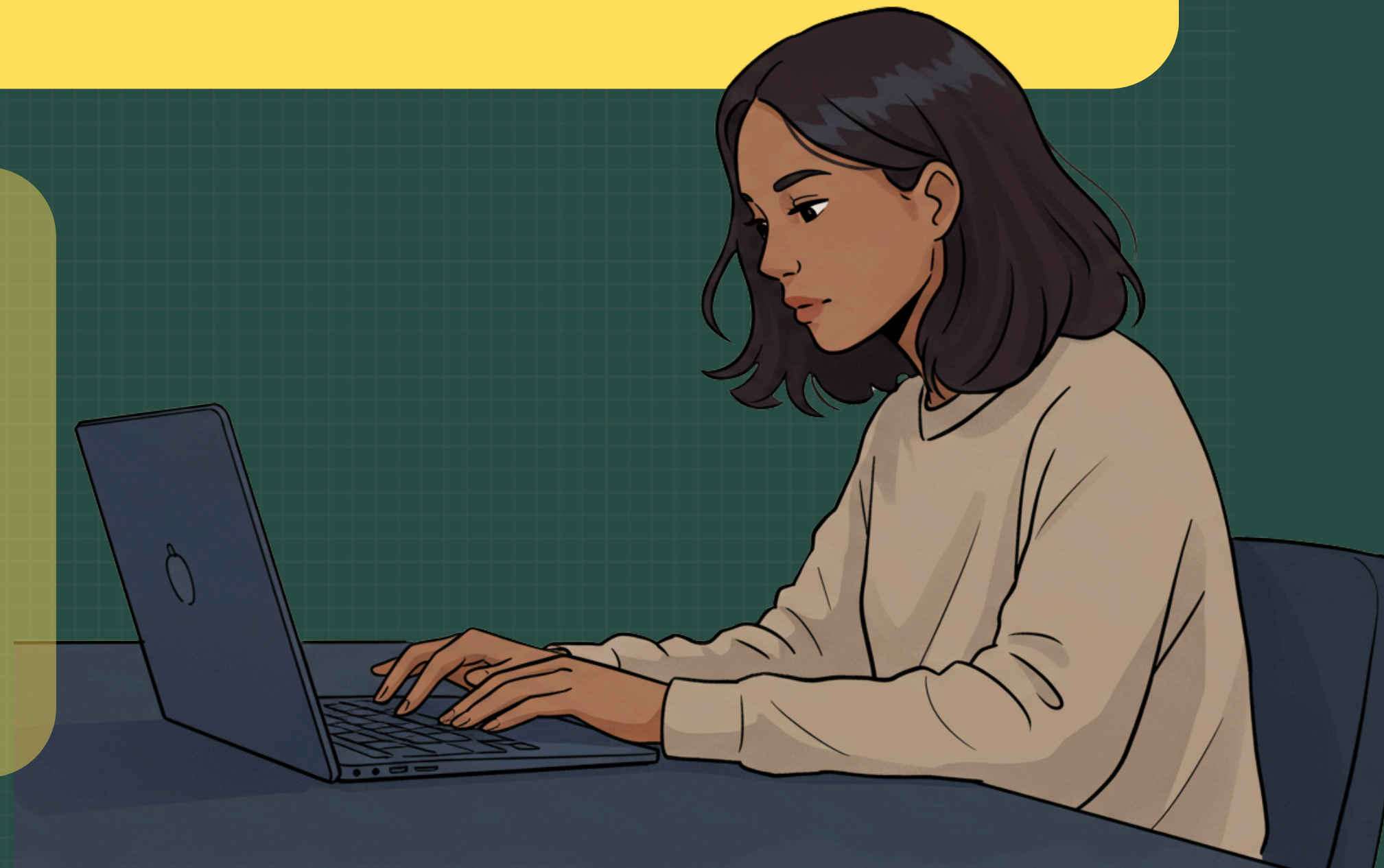
2º Forma (Optimizada): Iterar todos los números del 1 al $\sqrt{N}$ y verificar cuál de ellos es divisor de este número.

La complejidad de nuestro algoritmo pasa a ser: **$O(\sqrt{N})$**

Lo que nos permitirá analizar números más grandes.

FUNCIONES

Una función es un bloque de código reutilizable. Comúnmente recibe una entrada (parámetros), realiza un cálculo con ellos, y produce una salida.



FUNCIONES

El tipo de dato del
valor de retorno

```
tipoDeDato nombreFuncion(tipoDeDato parametro) {  
    // calculo entre los parámetros  
    return dato;  
}
```

valor que usaremos
para hallar un resultado

El resultado de realizar un cálculo
con los parámetros

FUNCIONES

El tipo de dato del
valor de retorno

```
int sumar(int a, int b) {  
    return (a + b);  
}
```

Valores que le pasamos
para realizar un cálculo de
suma y retornar el resultado

La suma de dos enteros retorna un entero
Por eso colocamos: int sumar(...)

FUNCIONES

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int a = 10, b = 12;
7      cout << "La primera suma es: " << (a + b) << endl;
8
9      int c = 15, d = 5;
10     cout << "La segunda suma es: " << (c + d) << endl;
11
12     /*
13      La primera suma es: 22
14      La segunda suma es: 20
15     */
16
17     return 0;
18 }
```



```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int sumar(int numero1, int numero2) {
6      int suma = numero1 + numero2;
7      return suma;
8  }
9
10 int main() {
11     int a = 10, b = 12;
12     cout << "La primera suma es: " << sumar(a, b) << endl;
13
14     int c = 15, d = 5;
15     cout << "La segunda suma es: " << sumar(c, d) << endl;
16
17     /*
18      La primera suma es: 22
19      La segunda suma es: 20
20     */
21
22     return 0;
23 }
```

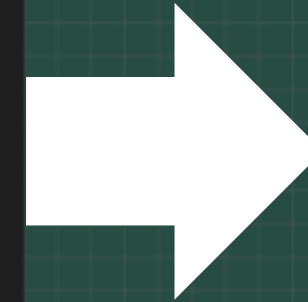
El tipo de dato del
valor de retorno

FUNCIONES

```
→ vector<int> separarPares(int a, int b, int c) {  
    → vector<int> pares;  
    if (a % 2 == 0) pares.push_back(a);  
    if (b % 2 == 0) pares.push_back(b);  
    if (c % 2 == 0) pares.push_back(c);  
  
    → return pares;  
}
```

FUNCIÓN PARA LA DIVISIBILIDAD

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      long long N = 10000000000000;
7
8      vector<long long> divisores;
9      for (long long div = 1; div * div ≤ N; div++) {
10         if (N % div == 0) {
11             divisores.push_back(div);
12             if (N / div ≠ div) {
13                 divisores.push_back(N / div);
14             }
15         }
16     }
17
18     cout << divisores.size() << endl;
19     for (int i = 0; i < divisores.size(); i++) {
20         cout << divisores[i] << " ";
21     }
22
23     return 0;
24 }
```



```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  vector<long long> obtenerDivisores(long long N) {
6      vector<long long> divisores;
7      for (long long div = 1; div * div ≤ N; div++) {
8          if (N % div == 0) {
9              divisores.push_back(div);
10             if (N / div ≠ div) {
11                 divisores.push_back(N / div);
12             }
13         }
14     }
15
16     return divisores;
17 }
18
19 int main() {
20     long long N = 10000000000000;
21     vector<long long> divisores = obtenerDivisores(N);
22
23     cout << divisores.size() << endl;
24     for (int i = 0; i < divisores.size(); i++) {
25         cout << divisores[i] << " ";
26     }
27
28     return 0;
29 }
```


Divisores del futuro

En el año 2147, la humanidad ya ha colonizado varios planetas y viaja por el espacio como si fuera un paseo en bicicleta. Pero hay un problema... A lo largo de los siglos, los humanos han aprendido por las malas que no todas las civilizaciones alienígenas son amigables.

Para nuestra buena suerte, sabemos que los números tal como los conocemos son una interpretación netamente humana y que otras especies podrían interpretarlos de formas completamente diferentes. Por ejemplo, nosotros los humanos tenemos como mínima unidad el 1. Pero para los Klorbaxianos, es 3.4623 aproximadamente.

Esto significa que nuestras matemáticas no tienen sentido para ellos, y aquí es donde la humanidad encontró la solución perfecta: "En lugar de comunicarnos con palabras, codificamos nuestros mensajes utilizando los divisores de un número N ."

Para facilitar la tarea de codificar nuestros mensajes, el Alto Mando de la Flota Terrestre te ha pedido crear un programa que halle la cantidad de divisores de un número N y que los muestre en orden ascendente.



<https://blitzcoding.com/problem-detail/93>

PRIMALIDAD

Los números primos son números enteros que solo se pueden dividir entre 1 y ellos mismos.



PRIMALIDAD

- Nuestro objetivo ahora es verificar si un entero positivo N es primo o no.
- Todo número primo tiene como únicos divisores al 1 y al mismo N .
- Si algún entero p cumple que $p \mid N$ entonces N no es primo, caso contrario N es primo.

TEST DE PRIMALIDAD I

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int N = 12;
7
8      for (int div = 2; div < N; div++) {
9          if (N % div == 0) {
10             cout << "No es primo";
11             return 0;
12         }
13     }
14     cout << "Sí es primo";
15
16     return 0;
17 }
```

1º Forma: Un primer enfoque al igual que en la sección anterior sería iterar todos los números $2 \leq \text{div} \leq N - 1$.

Desventaja: Si el número es muy grande, el programa nos puede dar **Time Limit Exceeded** por tener una complejidad algorítmica: **$O(N)$**

TEST DE PRIMALIDAD II

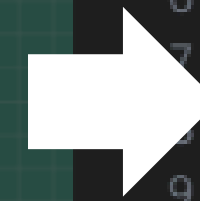
```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int N = 13;
7
8      for (int div = 2; div * div ≤ N; div++) {
9          if (N % div == 0) {
10             cout << "No es primo";
11             return 0;
12         }
13     }
14     cout << "Sí es primo";
15
16     return 0;
17 }
```

2º Forma: Basándonos en lo anterior, basta con buscar un divisor **div** iterando de 2 a $\sqrt{N}$. Si no se encuentra ningún divisor en ese intervalo entonces **N** es primo.

La complejidad de nuestro algoritmo se reduce a: **$O(\sqrt{N})$**

FUNCIÓN PARA EL TEST DE PRIMALIDAD

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int N = 13;
7
8      for (int div = 2; div * div ≤ N; div++) {
9          if (N % div == 0) {
10             cout << "No es primo";
11             return 0;
12         }
13     }
14     cout << "Sí es primo";
15
16     return 0;
17 }
```



```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  bool esPrimo(int N) {
6      for (int div = 2; div * div ≤ N; div++) {
7          if (N % div == 0) {
8              return false;
9          }
10     }
11     return true;
12 }
13
14 int main() {
15     int N = 13;
16
17     if (esPrimo(N) == true) {
18         cout << "Es primo.";
19     } else {
20         cout << "No es primo.";
21     }
22
23     return 0;
24 }
```


Test de primalidad (Fácil)

Dado un entero N determina si es primo o no.

Descripción de entrada:

La entrada consiste únicamente de un entero N .

Descripción de salida:

Imprime YES si el número es primo o NO en caso contrario.

Notas del ejercicio:

$2 \leq N \leq 10^{15}$

Se asegura que la cantidad de casos no excede 20.



<https://blitzcoding.com/problem-detail/43>

TEST DE PRIMALIDAD III

Test de primalidad (Difícil)

Dado un entero N determina si es primo o no.

Descripción de entrada:

La entrada en cada caso consiste únicamente de un entero N .

Descripción de salida:

Imprime YES si el número es primo o NO en caso contrario.

Notas del ejercicio:

$2 \leq N \leq 10^{18}$ ←

3º Forma: A diferencia de los anteriores métodos determinísticos. Existen **algoritmos probabilísticos** para determinar si un número es primo. Su complejidad es mucho menor, lo que nos permite analizar números mucho más grandes.

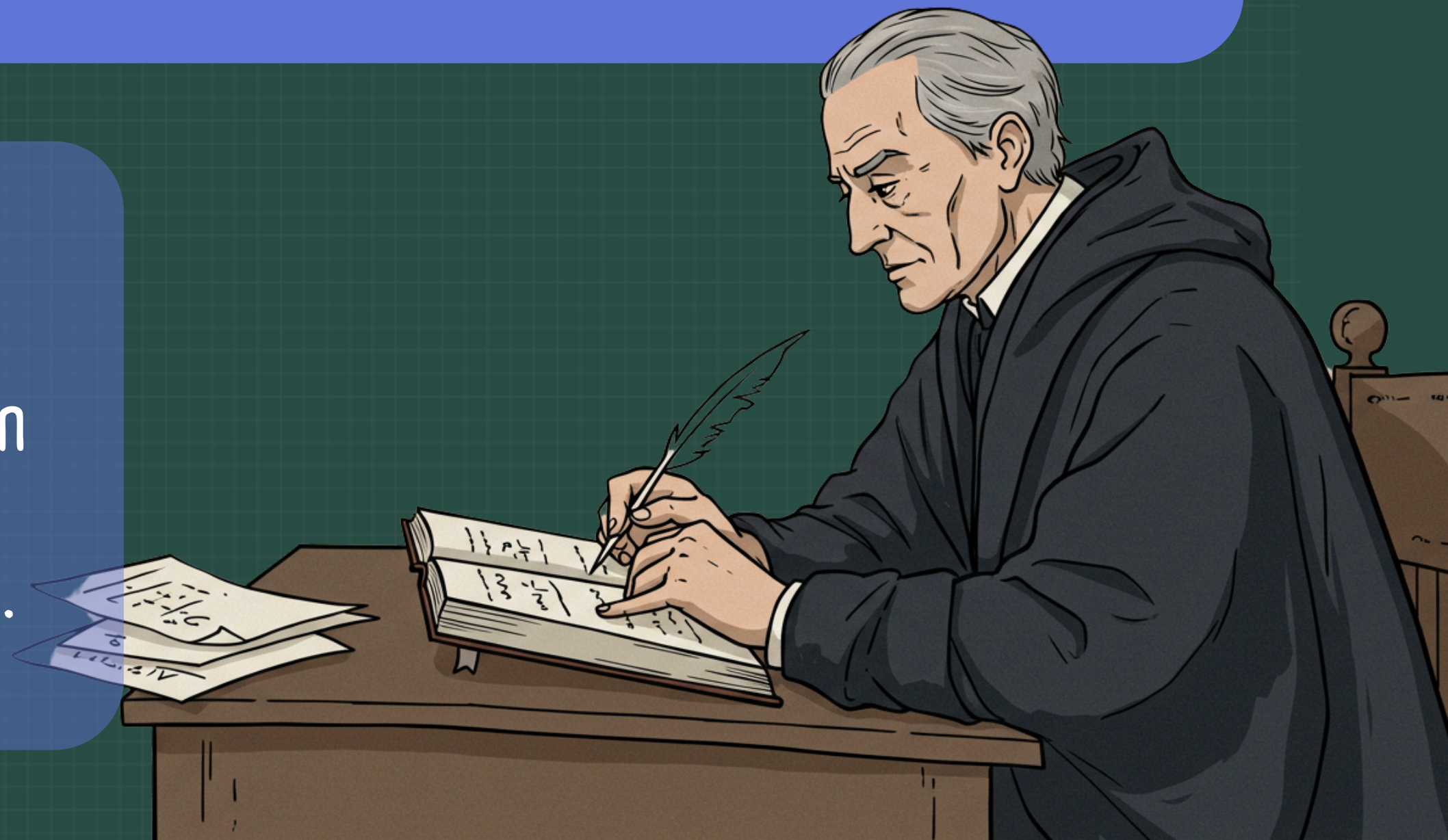
Algoritmos como: El test de Miller-Rabin, el test de Fermat, Baillie-PSW, etc.



<https://blitzcoding.com/problem-detail/44>

FACTORIZACIÓN

En aritmética, la factorización es la descomposición de un número en factores, es decir, en números que multiplicados entre sí dan el número original.



FACTORIZACIÓN

- Sea N un entero positivo, lo que se busca es obtener la descomposición de dicho número en sus factores primos, por ejemplo:

$$60 = 2^2 \cdot 3^1 \cdot 5^1$$

- El objetivo en nuestro algoritmo sería obtener todas las parejas:
 - { factor primo, exponente }
- Por ejemplo, para el 60 tendríamos las siguientes: {2, 2}, {3, 1}, {5, 1}

- Generalizando el caso anterior, tendríamos algo como:

$$N = p_1^{exp_1} \cdot p_2^{exp_2} \cdots p_k^{exp_k}$$

- Usando nuestras conclusiones pasadas, podemos notar que todo **factor primo** (que no deja de ser un divisor de N) se encuentra en el intervalo de: $2 \leq p \leq \sqrt{N}$
- Primero buscamos **un divisor de N** . Diremos que este es un **factor primo**.
- Para hallar su **exponente**, tenemos que dividir N entre p tantas veces como sea posible actualizando el valor de N por el resultado de la división, **el exponente será la cantidad de veces que se ejecutó la división**.

FACTORIZACIÓN

```
1  int N = 60;
2
3  vector<pair<int, int>> factores;
4  if (N == 1) {
5      cout << "1 1";
6      return 0;
7  }
8
9  for (int p = 2; p * p <= N; p++) {
10     if (N % p == 0) {
11         int exp = 0;
12         while (N % p == 0) {
13             exp++;
14             N /= p;
15         }
16         factores.push_back({ p, exp });
17     }
18 }
```

Si el número N es igual a 1, entonces la única pareja que deberíamos obtener es $\{1, 1\}$.

Si N es diferente de 1, buscamos un factor primo p , su exponente será la cantidad de veces que se puede dividir N entre p .

Después de tener $\{p, \text{exp}\}$, almacenamos la pareja en un vector y buscamos la siguiente hasta que N se quede en 1.


```
1  int N = 60;
2
3  vector<pair<int, int>> factores;
4  if (N == 1) {
5      cout << "1 1";
6      return 0;
7  }
8
9  for (int p = 2; p * p ≤ N; p++) {
10     if (N % p == 0) {
11         int exp = 0;
12         while (N % p == 0) {
13             exp++;
14             N /= p;
15         }
16         factores.push_back({ p, exp });
17     }
18 }
```

FACTORIZACIÓN

```
1  int N = 60;
2
3  vector<pair<int, int>> factores;
4  if (N == 1) {
5      cout << "1 1";
6      return 0;
7  }
8
9  for (int p = 2; p * p <= N; p++) {
10     if (N % p == 0) {
11         int exp = 0;
12         while (N % p == 0) {
13             exp++;
14             N /= p;
15         }
16         factores.push_back({ p, exp });
17     }
18 }
```

Con la lógica anterior, cuando $N = 60$, esperaríamos las siguientes parejas: $\{2, 2\}$, $\{3, 1\}$, $\{5, 1\}$.

Pero en su lugar, solo obtenemos el siguiente resultado: $\{2, 2\}$, $\{3, 1\}$

Esto ocurre porque en la última iteración, tenemos que $N' = 5$ y $p = 4$. Lo cual no cumple la condición del bucle $4 \times 4 \leq 5$.

FACTORIZACIÓN

```
1  int N = 60;
2
3  vector<pair<int, int>> factores;
4  if (N == 1) {
5      cout << "1 1";
6      return 0;
7  }
8
9  for (int p = 2; p * p <= N; p++) {
10     if (N % p == 0) {
11         int exp = 0;
12         while (N % p == 0) {
13             exp++;
14             N /= p;
15         }
16         factores.push_back({ p, exp });
17     }
18 }
19 if (N != 1) factores.push_back({ N, 1 });
```

Para evitar que queden factores primos sobrantes, verificamos si N' es un valor diferente de 1, y si lo es, lo agregamos como factor primo: $\{N', 1\}$

NOTA: Este comportamiento solo ocurre si el exponente del último factor primo es 1. Por ejemplo, para el número 18 no ocurre este problema porque su factor final tiene exponente 2: $\{2, 1\} \{3, 2\}$.

FUNCIÓN PARA LA FACTORIZACIÓN

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  vector<pair<int, int>> factorizacion(int N) {
6      vector<pair<int, int>> factores;
7      if (N == 1) {
8          factores.push_back({ 1, 1 });
9          return factores;
10     }
11
12     for (int p = 2; p * p ≤ N; p++) {
13         if (N % p == 0) {
14             int exp = 0;
15             while (N % p == 0) {
16                 exp++;
17                 N /= p;
18             }
19             factores.push_back({ p, exp });
20         }
21     }
22     if (N != 1) factores.push_back({ N, 1 });
23
24     return factores;
25 }
```

```
1  int main() {
2      int N = 18;
3      vector<pair<int, int>> factores = factorizacion(N);
4
5      /* Imprimir las parejas */
6      for (int i = 0; i < factores.size(); i++) {
7          cout << factores[i].first << " " << factores[i].second << endl;
8      }
9
10     return 0;
11 }
```

Factorización prima (fácil)

Dado un número entero n , calcule sus factores primos (con repeticiones si es necesario).

Descripción de entrada:

La primera línea de entrada siempre indica el número de casos de prueba del problema.

La primera línea de cada caso de prueba contiene un entero n ($1 \leq n \leq 10^{12}$) — El entero a factorizar.

Descripción de salida:

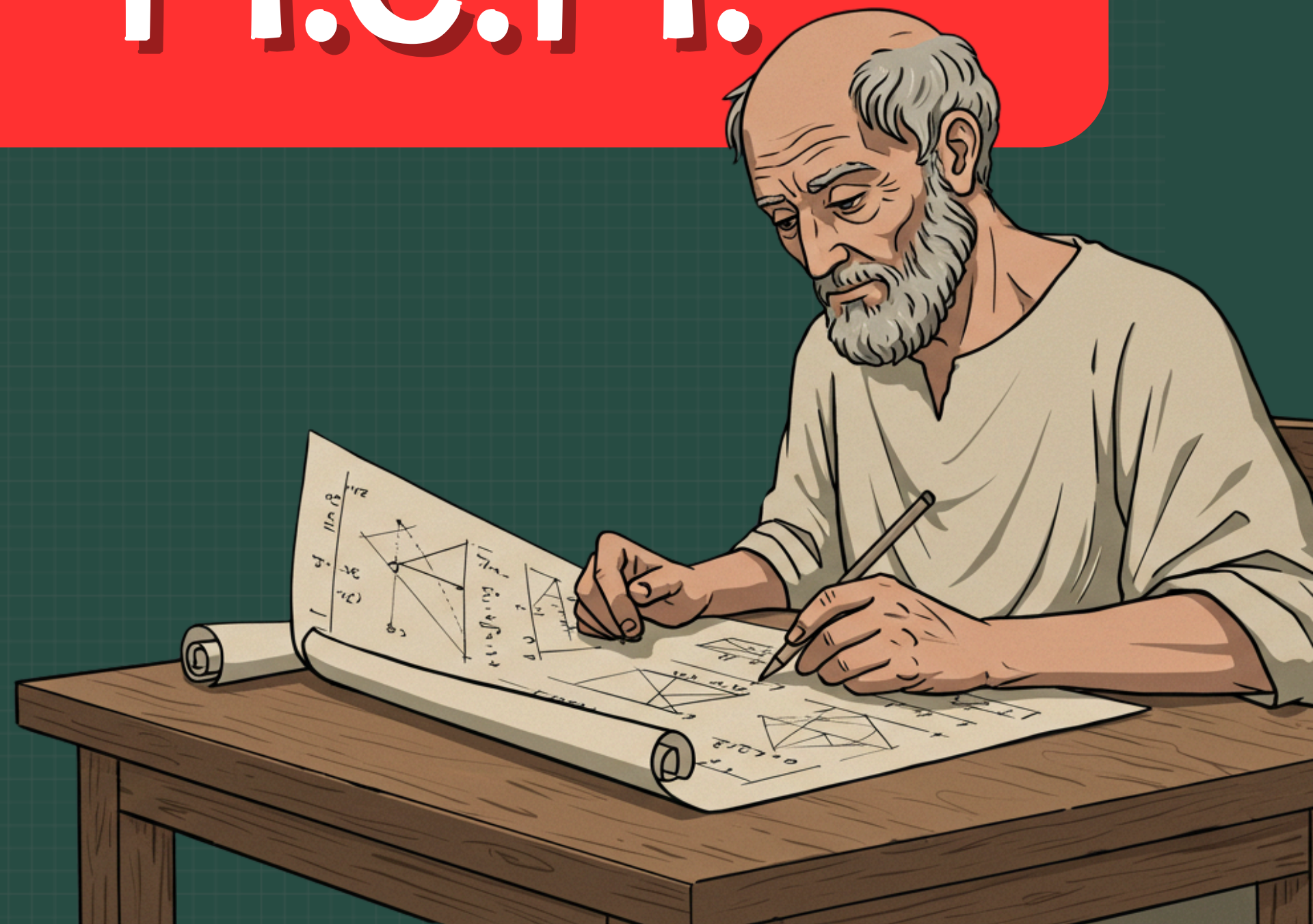
Imprima una sola línea con los factores primos de n en orden ascendente.



<https://blitzcoding.com/problem-detail/94>

M.C.D. - M.C.M.

Máximo Común Divisor (M.C.D.)
y Mínimo Común Múltiplo
(M.C.M.)



M.C.D.

- El **Máximo Común Divisor** (Greatest Common Divisor en inglés) de dos números, es aquel **divisor común** de ambos números **con mayor valor posible**.
- Por ejemplo, veamos todos los **divisores** de 30 y 48:
 - $30 \rightarrow \{ 1, 2, 3, 5, 6, 10, 15, 30 \}$
 - $48 \rightarrow \{ 1, 2, 3, 4, 6, 8, 12, 16, 24, 48 \}$
- De todos sus divisores **comunes**, el **mayor** es **6**. De aquí podemos concluir que el $M.C.D.(30, 48) = 6$.

M.C.M.

- El Mínimo Común Múltiplo (Least Common Multiple en inglés) de dos números es aquel **múltiplo común** de ambos con el **menor valor posible**.
- Por ejemplo, veamos algunos de los múltiplos de 4 y 6:
 - $4 \rightarrow \{ 4, 8, 12, 16, 20, 24, 28, 32, 36, \dots \}$
 - $6 \rightarrow \{ 6, 12, 18, 24, 30, 36, \dots \}$
- De todos sus múltiplos **comunes**, el **menor** es 12. De aquí viene que el $M.C.M.(4, 6) = 12$.

GREATEST COMMON DIVISOR

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      // Para C++ 11 no hay una función de gcd
7      cout << "😭" << endl;
8
9      // C++ 14, C++ 17, en adelante
10     cout << __gcd(30, 48) << endl;
11
12     // Desde C++ 17 en adelante
13     cout << gcd(30, 48) << endl;
14
15     return 0;
16 }
```

LEAST COMMON MULTIPLE

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      // Para C++ 11 y C++14 NO
7      // hay una función de lcm
8      cout << "😭" << endl;
9
10     // Desde C++ 17 en adelante
11     cout << lcm(4, 6) << endl;
12
13     return 0;
14 }
```


Si la página nos obliga a usar C++ 11, deberemos crear la función del M.C.D. nosotros mismos. Lo haremos mediante el **Algoritmo de Euclides**. Además, tendremos en cuenta que para cualquier par de números a y b , se cumple que $a \times b = \text{lcm}(a, b) \times \text{gcd}(a, b)$. Por lo que también tenemos el M.C.M.

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  // Creamos la función nosotros mismos
6  int gcd(int a, int b) {
7      if (a > b) swap(a, b);
8      if (a == 0) return b;
9      return gcd(b % a, a);
10 }
11
12 int main() {
13     cout << gcd(30, 48);
14
15     return 0;
16 }
```

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int gcd(int a, int b) {
6      if (a > b) swap(a, b);
7      if (a == 0) return b;
8      return gcd(b % a, a);
9  }
10
11 // Creamos la función nosotros mismos
12 int lcm(int a, int b) {
13     return (a * b) / gcd(a, b);
14 }
15
16 int main() {
17     cout << lcm(4, 6);
18
19     return 0;
20 }
```

GRACIAS POR SU ATENCIÓN

by: Zahir La Rosa

